



目录

CONTENTS



课题背景



问题探究与方案提出



设计与实现

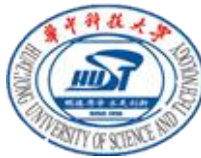


测试与评估



01 课题背景

- ✓ 课题背景与研究趋势
- ✓ 用忆阻器阵列进行MVM运算



课题背景与研究趋势

■ 高性能线性系统求解

- 主流方法：迭代法，由CG/BICGSTAB等算法实现
- 需要优化其关键内核：矩阵向量乘法（MVM）运算

■ 基于忆阻阵列进行MVM运算的优势

- 低数据迁移消耗
- 高并行性

■ 研究趋势

- 高精度浮点数格式的部署与低开销的矩阵映射方式
- 忆阻阵列MVM的低能耗计算模式

用忆阻器阵列进行MVM运算

忆阻阵列运算过程

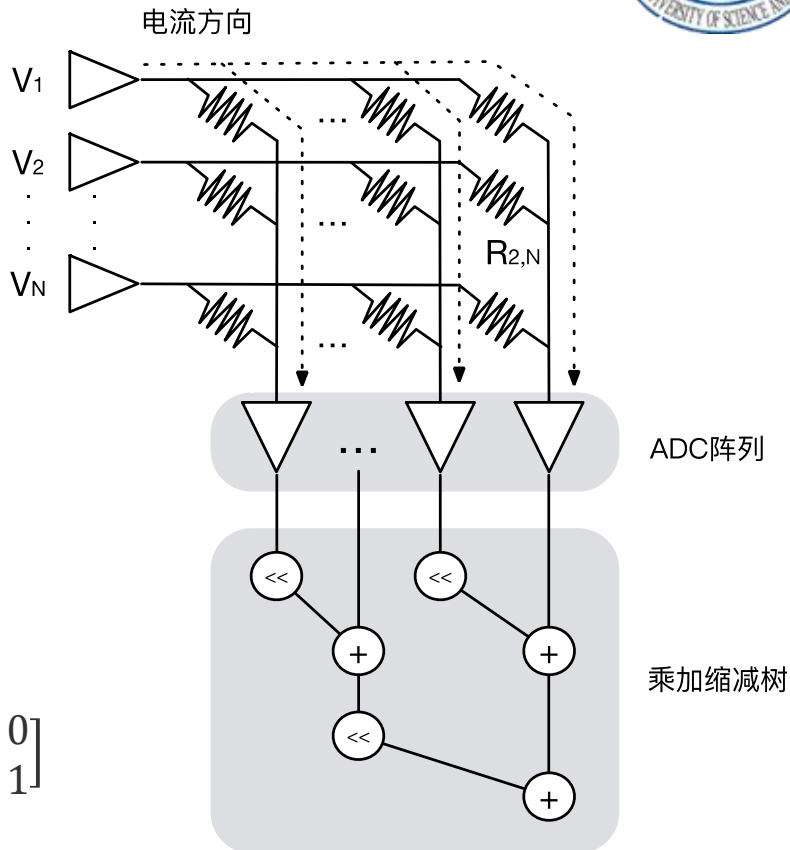
- 需要求解: $Ax = b$
- 映射关系: $a_{ij} = \frac{1}{R_{ji}}, x_i = V_i$
- 求解过程: $b_j = \sum_{i=1}^n \frac{v_i}{R_{ij}}$

分片策略

- 解决模拟器件精度不够的问题

$$\bullet \begin{bmatrix} 1 & 6 \\ 2 & 5 \end{bmatrix} = 2^2 \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} + 2^1 \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} + 2^0 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- 不同向量位片用乘加缩减树进行整合





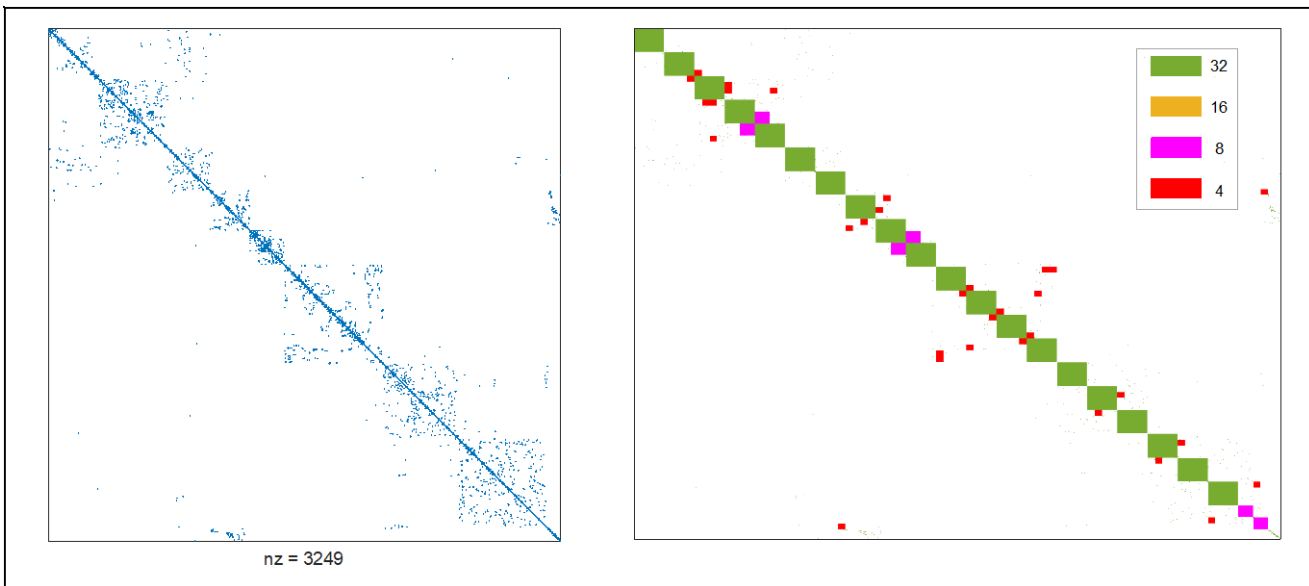
02 问题探究与方案提出

- ✓ 稀疏矩阵的低开销映射模式
- ✓ 高精度浮点数的部署方式
- ✓ 为不同精度的求解应用优化能量开销
- ✓ 叶子结点数可变的乘加缩减树

稀疏矩阵的低开销映射方式

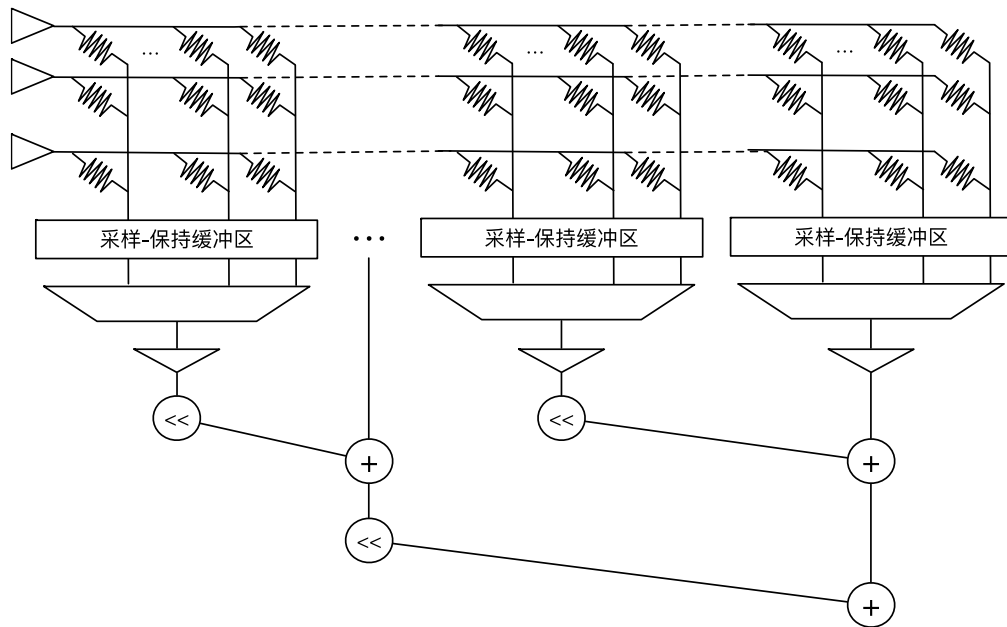


- 问题1：科学计算中矩阵的稀疏性
- 方案：异构分块映射的方式



高精度浮点数的部署方式

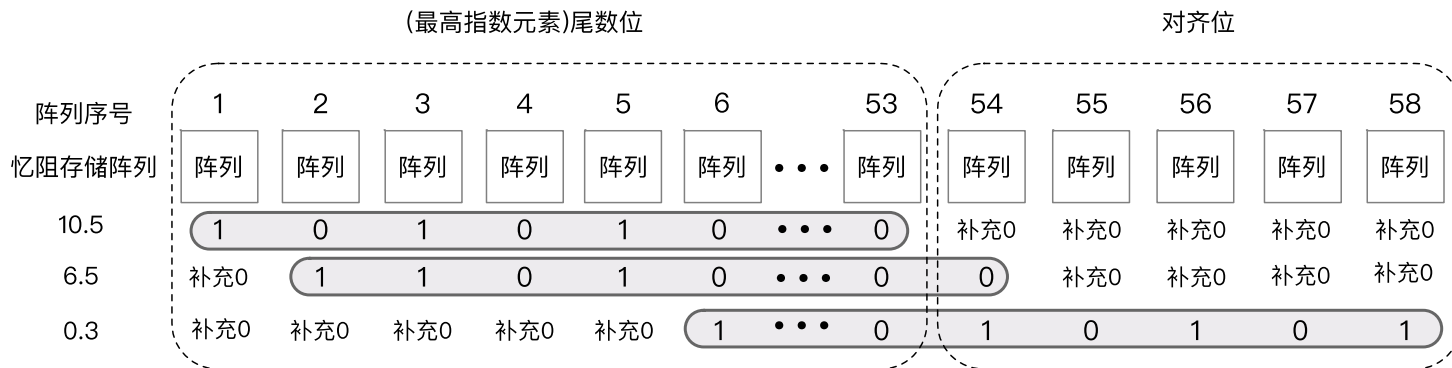
- 问题2：现有的忆阻器阵列无法支持以高精度浮点数为主的科学计算
- 方案：IEEE-754双精度浮点数尾数的位片映射



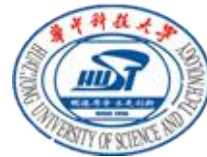
高精度浮点数的部署方式

■ 将矩阵块映射到存储簇上

➤ 存储簇中对齐位阵列的概念与对齐位的部署

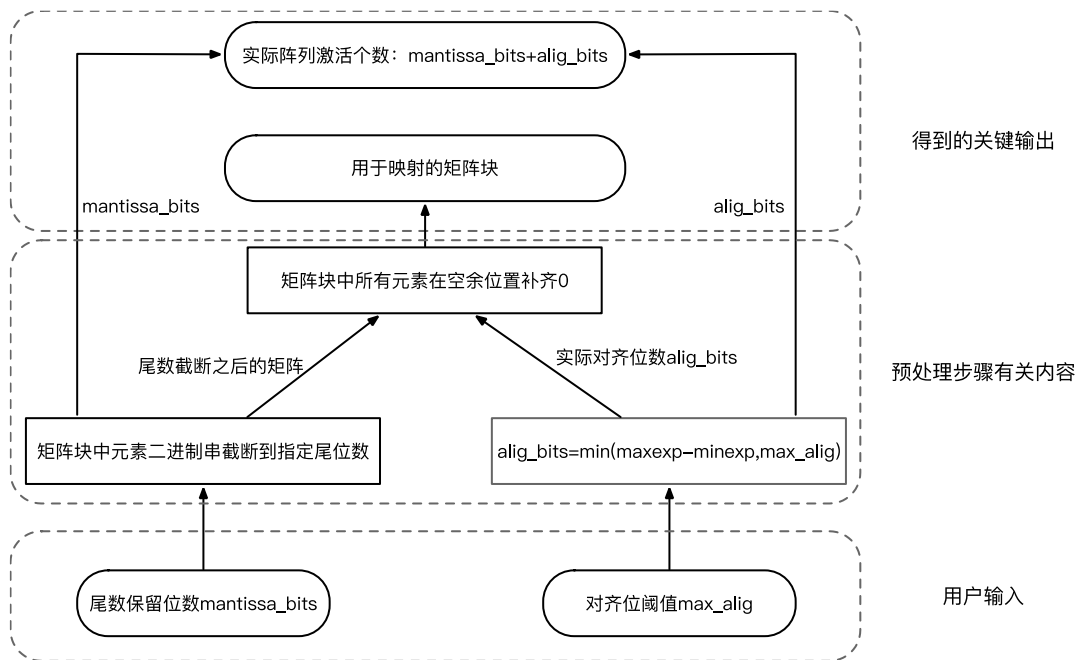


注：10.5、6.5、0.3这三个浮点数对应的二进制指数分别为：3、2、-2。



为不同求解精度的应用优化能量开销

- 问题3：对于低求解精度应用，高精度计算模式带来能量浪费
- 问题4：固定的对齐位模式带来潜在能量浪费
- 方案：尾数压缩与对齐位优化策略



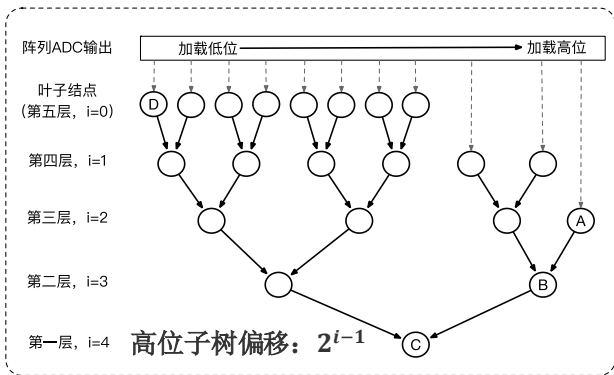
叶子结点数可变的流水乘加缩减树

■ 叶子结点数可变带来的问题

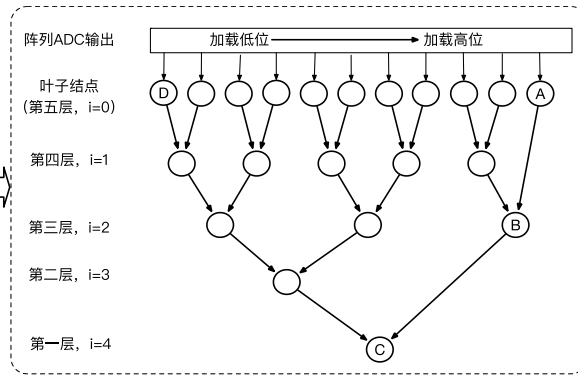
- 移位策略失效
- 流水计算出错

■ 解决方式

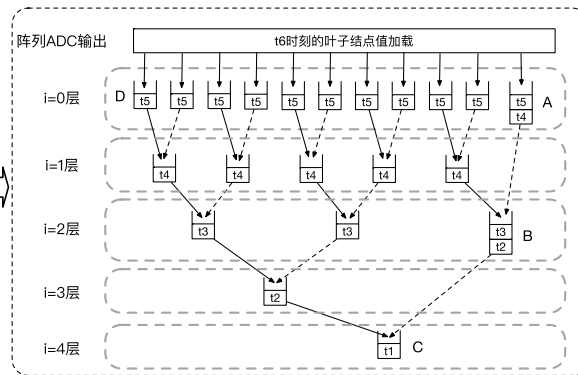
- 从叶子结点开始构造树的方式
- 维护一个先进先出的变长队列



传统的拥有错误移位计算模式的树的构造



叶子结点数可变的流水乘加缩减树 (正确形式)



流水乘加缩减树的存储队列结构

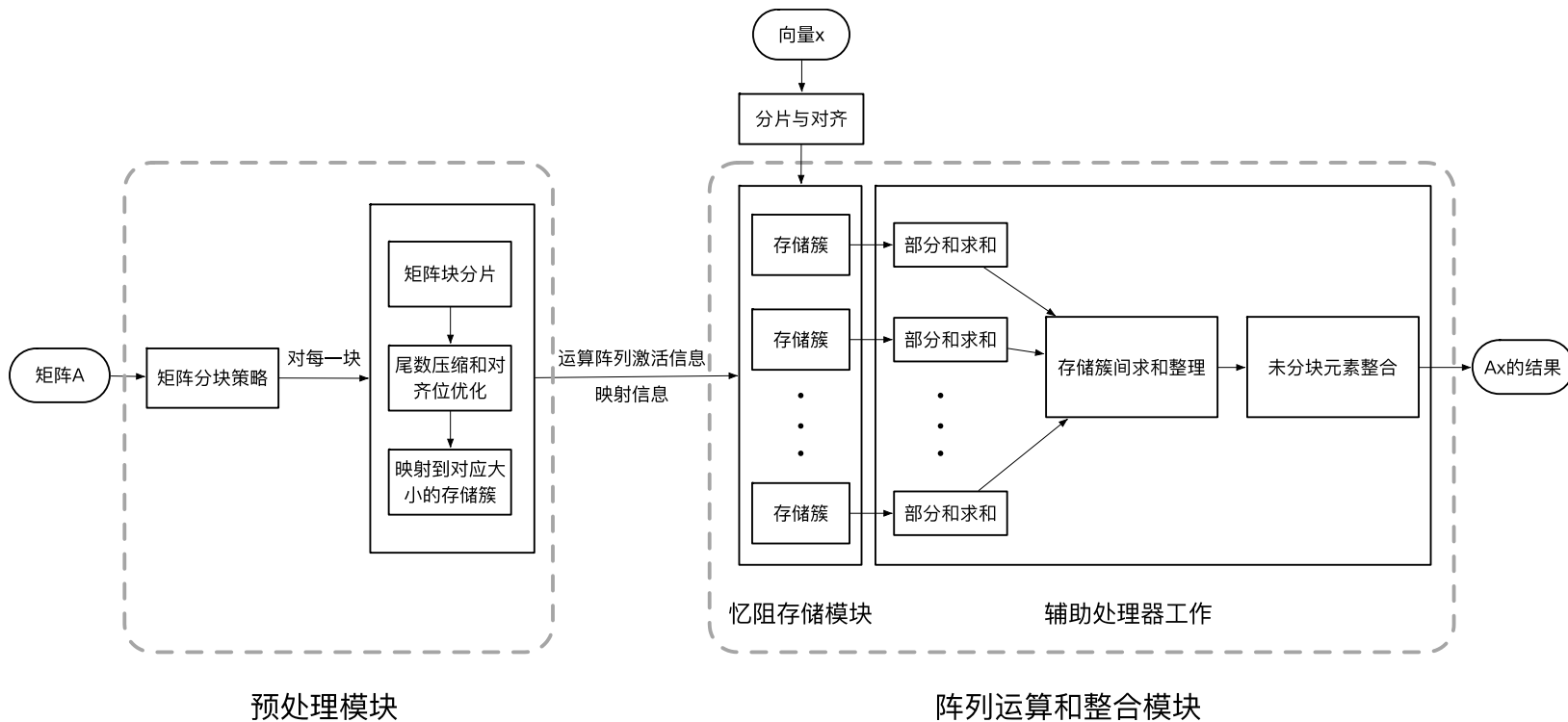
父结点整合计算方式: $node.val.push(node.val.left.pop + node.val.right.pop \ll 2^{i-1})$



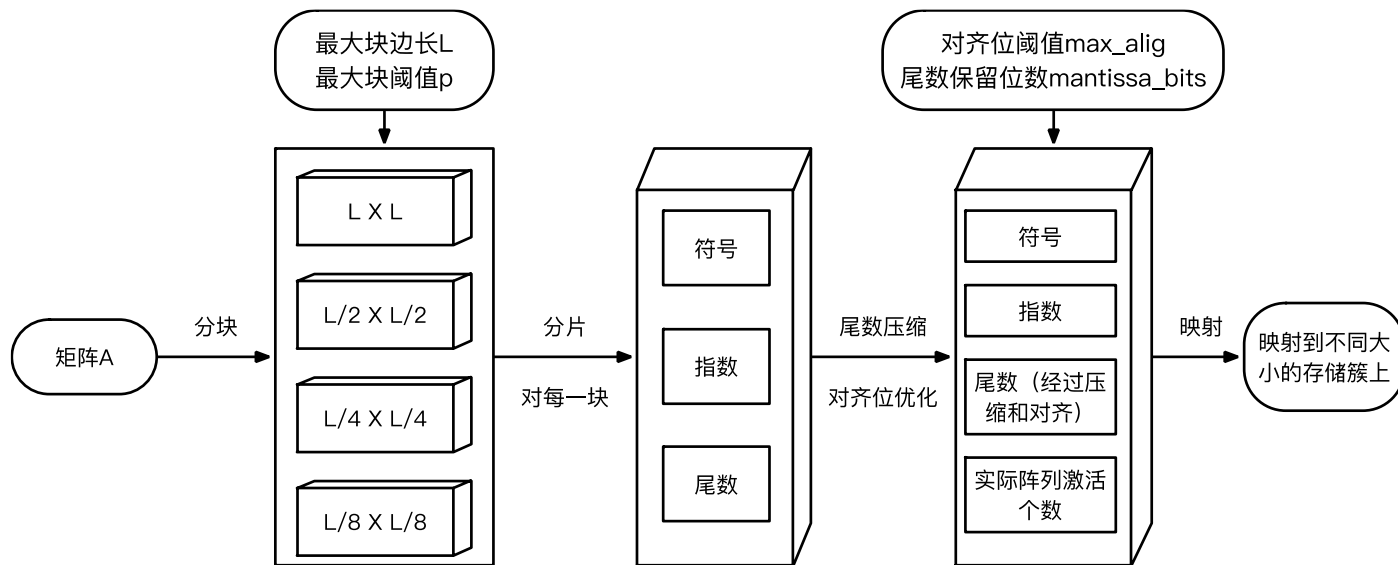
03 设计与实现

- ✓ 系统总体结构设计
- ✓ 预处理模块
- ✓ 阵列运算与整合模块
- ✓ 辅助处理器中的运算流程

基于异构阵列集合的自选尾数压缩系统



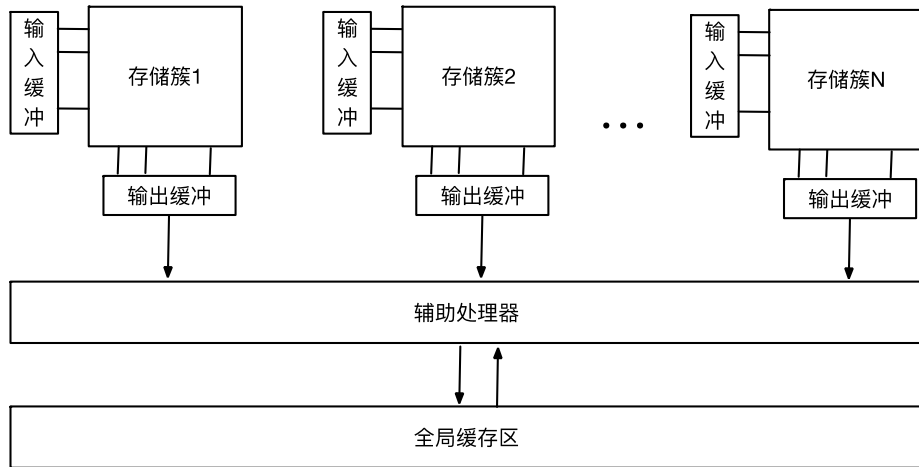
预处理模块



位片式异构映射策略、尾数压缩与对齐位优化策略

阵列运算与整合模块

■ 整体结构



- 输入缓冲：向量元素以时序位片形式加载到阵列上。
- 输出缓冲：阵列运算的结果。
- 辅助处理器与全局缓冲区：用于整合以及储存中间结果和最后结果。

■ 负数处理

- 矩阵负数：正数与负数分别映射到两组不同的阵列上。
- 电压/电流负数：电压本身可以为负数。电流转换时进行值为N的偏移，将电流范围 $[-N, N]$ 转换为 $[0, 2N]$ 。

辅助处理器中的运算流程

- 整合同一存储簇中不同向量位片的结果

$$b''_{cposk}(b''_{cnegk}) = \sum_j A_k \times x_j$$

- 存储簇正负阵列集合的结果相减

$$b''_{ck} = b''_{cposk} - b''_{cnegk}$$

- 二进制转浮点数

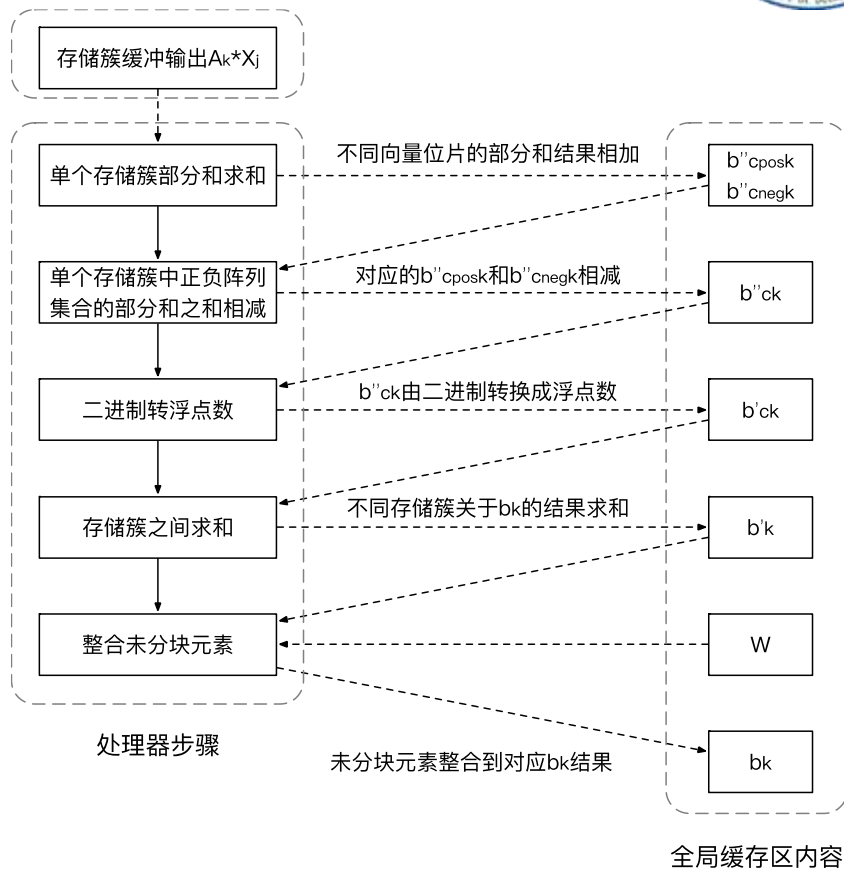
$$b''_{ck} \rightarrow b'_{ck}$$

- 存储簇间整合

$$b'_k = \sum_c b'_{ck}$$

- 整合未分块元素

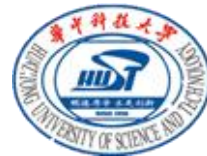
$$b_k = b'_k + \sum_i W_i * x[W_{ix}] \quad (W_{iy} = k)$$





04 测试与评估

- ✓ 低开销映射的测试与评估
- ✓ 不同压缩策略下的求解精度
- ✓ 不同压缩策略下的运算阵列能耗
- ✓ 不同压缩策略下的ADC能耗
- ✓ 不同压缩策略下的能耗评估总结



低开销映射的测试与评估

■ 评估方法

- 阵列面积占比

$$Area_{\text{异构}} / Area_{\text{直接映射}}$$

- 非零元素密度比例

$$\frac{NNZ_{\text{异构}} / Area_{\text{异构}}}{NNZ_{\text{直接映射}} / Area_{\text{直接映射}}}$$

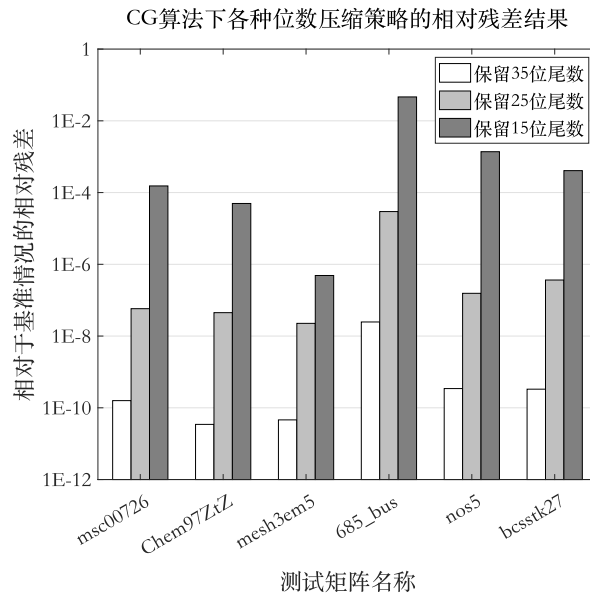
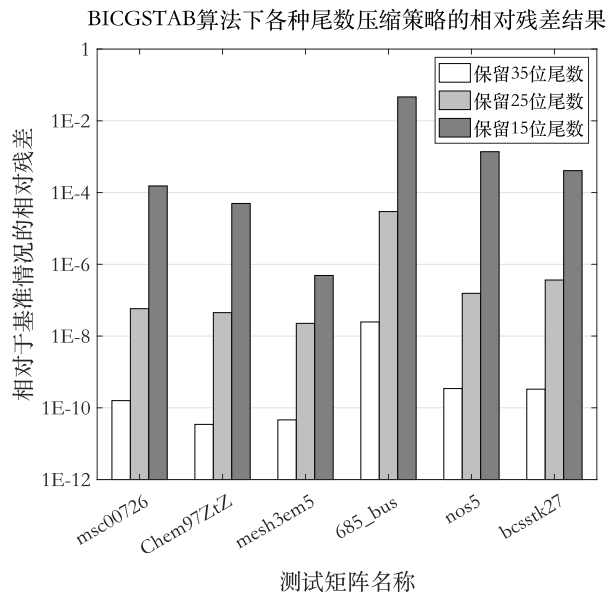
■ 评估结果

相对于直接映射方案而言各评估量的比例

矩阵名称	阵列面积占比	非零元素密度比例
msc00726	50.51%	1.98
Chem97ZtZ	11.20%	8.93
mesh3em5	22.07%	4.53
685_bus	11.16%	8.96
nos5	33.90%	2.95
bcsstk27	15.04%	6.65
平均值	23.98%	5.67

不同压缩策略下的求解精度

不同压缩策略下的求解精度



➤ 经过对数平均，三种压缩策略相对于软件求解基线的残差分别约为 10^{-9} 、 10^{-7} 、 10^{-3} 数量级。

不同压缩策略下的运算阵列能耗

评估标准

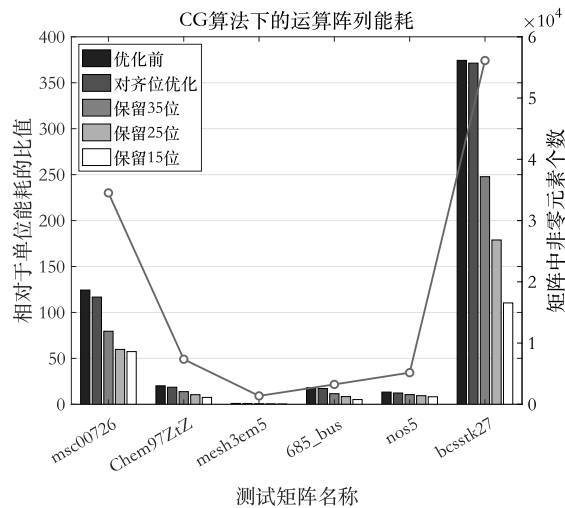
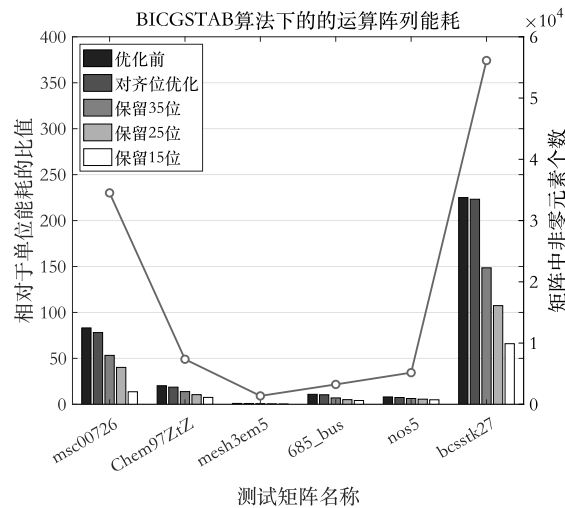
➤ 每个存储单元的能耗

➤ 总能耗

$$P = \begin{cases} \frac{V_{read}^2}{R_{on}} & (\text{映射值为 } 1) \\ \frac{V_{read}^2}{R_{off}} & (\text{映射值为 } 0) \end{cases}$$

$$E_{Crossbar} = \sum_{\text{所有Ax计算}} \sum_{\text{所有存储簇}} \sum_{\text{所有向量位片}} \sum_{\text{所有阵列}} \sum_{\text{所有存储单元}} P \log_2 N$$

评估结果



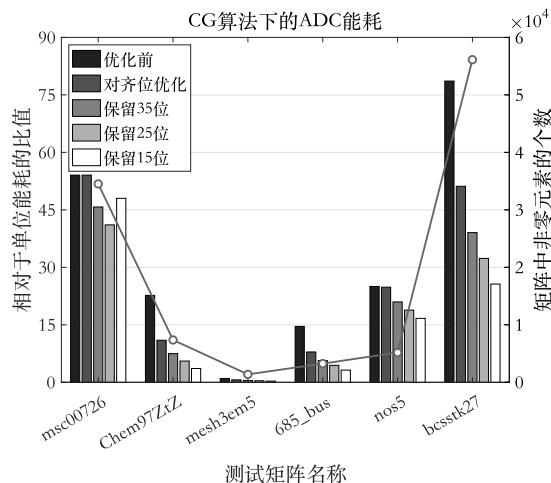
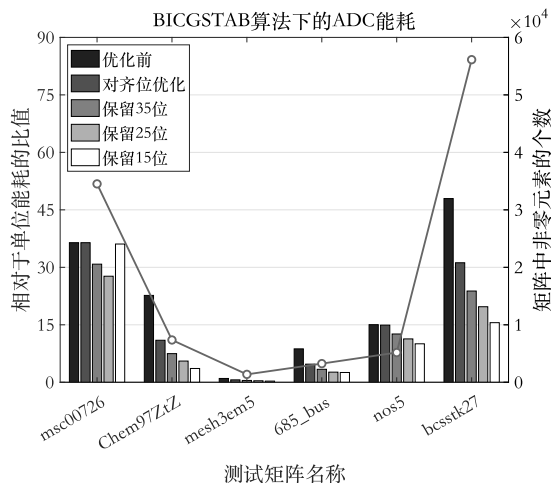
- NNZ ↑, 能耗 ↑
- 激活阵列个数 ↓, 能耗 ↓
- 迭代次数 ↑, 能耗 ↑。

不同压缩策略下的ADC能耗

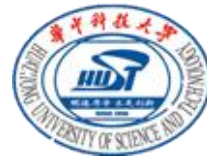
评估标准

- 存储簇一次ADC: $M \times N \times N \log_2 N$ 成比例
- 计算能耗: $E_{ADC} = \sum_{\text{所有AX计算}} \sum_{\text{所有存储簇}} \sum_{\text{所有向量位片}} M \times N^2 \log_2 N$

评估结果



- NNZ ↑, 能耗 ↑
- 激活阵列个数 ↓, 能耗 ↓
- 迭代次数 ↑, 能耗 ↑。
- 保留位数过少, 能耗可能反常多。



不同压缩策略下能耗测试总结

运算阵列能耗平均结果

残差为 $0 \sim 10^{-3}$ 数量级时，运算阵列能耗约为优化前的95%~35%（即能耗减少了5%~65%）。

相对于未应用任何策略情况下的平均能耗比例（运算阵列能耗）

算法	对齐位优化	保留 35 位	保留 25 位	保留 15 位
相对残差	0	10^{-9}	10^{-7}	10^{-3}
BICGSTAB	94.74%	66.45%	50.84%	34.33%
CG	94.72%	66.57%	50.91%	37.82%

ADC能耗平均结果

残差为 $0 \sim 10^{-3}$ 数量级时，ADC能耗约为优化前的70%~45%（即能耗减少了30%~55%）。

相对于未应用任何策略情况下的平均能耗比例（ADC 能耗）

算法	对齐位优化	保留 35 位	保留 25 位	保留 15 位
相对残差	0	10^{-9}	10^{-7}	10^{-3}
BICGSTAB	72.34%	56.94%	48.32%	46.45%
CG	71.71%	56.33%	47.77%	42.77%